AIAC Assignment-19.2

Name: G.Harika

Ht.no:2303A51612

Batch.no:05

Task-1

Prompt:

Convert the following Python file handling program into Java. Ensure proper file reading and writing, exception handling, and same output behavior. Maintain clean formatting and equivalent logic.

Python Code:

Ass-19.2 > ass-19.2.py > process_file

```python
def process_file():
    try:
        with open("Ass-19.2/input.txt", "r") as infile:
            content = infile.read()
        print("Original Content:")
        print(content)
        modified_content = content.upper()
        with open("Ass-19.2/output.txt", "w") as outfile:
            outfile.write(modified_content)
        print("\nModified content written to output.txt")
    except FileNotFoundError:
        print("Error: input.txt not found.")
    except Exception as e:
        print("An error occurred:", e)
if __name__ == "__main__":
    process_file()
```

Output:

Ass-19.2
- ass-19.2.py
- input.txt
- output.txt
- food-api
- Lab.html
- New folder

```
Original Content:
Hello World
This is a sample file
AI Lab Practice

Modified content written to output.txt
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL LAB\AI ASSISTANT CODING>
```

Ass-19.2 > output.txt

```
HELLO WORLD
THIS IS A SAMPLE FILE
AI LAB PRACTICE
```

Java code:

```java
Ass-19.2 > FileProcessor.java > FileProcessor > main(String[])
import java.io.*;
public class FileProcessor {
    public static void processFile() {
        try {
            BufferedReader reader = new BufferedReader(new FileReader(fileName: "Ass-19.2/input.txt"));
            StringBuilder content = new StringBuilder();
            String line;

            while ((line = reader.readLine()) != null) {
                content.append(line).append(str: "\n");
            }
            reader.close();
            System.out.println(x: "Original Content:");
            System.out.println(content.toString());
            String modifiedContent = content.toString().toUpperCase();
            BufferedWriter writer = new BufferedWriter(new FileWriter(fileName: "Ass-19.2/output1.txt"));
            writer.write(modifiedContent);
            writer.close();
            System.out.println(x: "\nModified content written to output1.txt");
        } catch (FileNotFoundException e) {
            System.out.println(x: "Error: input.txt not found.");
        } catch (IOException e) {
            System.out.println("I/O Error: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("An unexpected error occurred: " + e.getMessage());
        }
    }
    Run|Debug
    public static void main(String[] args) {
        processFile();
    }
}
```

Output:

```
Original Content:
Hello World
This is a sample file
AI Lab Practice


Modified content written to output1.txt
```

Explanation:

The Python program reads content from a file (input.txt), displays it, converts the text to uppercase, and writes it to another file (output.txt). It uses with open() for automatic file handling and includes exception handling for missing files and general errors.

The Java version replicates the same functionality using: BufferedReader for reading files ,BufferedWriter for writing files ,StringBuilder for efficient string handling ,try-catch blocks for handling exceptions like FileNotFoundException and IOException

Both programs:

- Read the same input file
- Produce identical console output
- Generate the same modified output file

Task-2

Convert the following C++ sorting program (Bubble Sort) into an equivalent Python program. Ensure the same logic, correctness, and output behavior. Include proper input/output handling.

C++ Code:

main.cpp

```cpp
#include <iostream>
using namespace std;
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (arr[j] > arr[j+1]) {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}
int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    bubbleSort(arr, n);
    cout << "Sorted array:\n";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
```


```cpp
    cout << "Sorted array:\n";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
```


```cpp
    return 0;
}
```

Output:

```
Output

Enter number of elements: 5
Enter elements:
2 9 4 1 0
Sorted array:
0 1 2 4 9

=== Code Execution Successful ===
```

Python Code:

```python
def bubble_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                # swap
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr


def main():
    try:
        n = int(input("Enter number of elements: "))

        arr = list(map(int, input("Enter elements separated by space:\n").split()))

        if len(arr) != n:
            print("❌ Number of elements mismatch.")
            return

        sorted_arr = bubble_sort(arr)

        print("Sorted array:")
        print("sorted_arr)

    except ValueError:
        print("❌ Invalid input. Please enter integers only.")


if __name__ == "__main__":
    main()
```

Output:

```
Enter number of elements: 5
Enter elements separated by space:
1 9 4 2 0
Sorted array:
0 1 2 4 9
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL LAB\AI ASSISTANT CODING>
```

Explanation:

The original C++ program implements the **Bubble Sort algorithm**, where adjacent elements are repeatedly compared and swapped if they are in the wrong order. This process continues until the array is sorted.

The Python version follows the same logic: Uses nested loops to compare adjacent elements,Swaps elements using Python's tuple assignment ,Takes input dynamically and validates it ,Ensures the number of elements matches user input Both implementations:Perform the same sorting steps ,Produce identical sorted output ,Maintain algorithm correctness

Task-03

Prompt:

Convert the following SQL query with GROUP BY and aggregate functions into equivalent Pandas code. Ensure correct grouping, aggregation, and matching output.

SQL CODE& Output:

```sql
-- Create table
CREATE TABLE sales_data (
    department TEXT,
    sales INTEGER
);

-- Insert data
INSERT INTO sales_data VALUES ('HR', 1000);
INSERT INTO sales_data VALUES ('IT', 2000);
INSERT INTO sales_data VALUES ('HR', 1500);
INSERT INTO sales_data VALUES ('Sales', 3000);
INSERT INTO sales_data VALUES ('IT', 2500);
INSERT INTO sales_data VALUES ('Sales', 3500);

-- Query (GROUP BY + SUM)
SELECT department, SUM(sales) AS total_sales
FROM sales_data
GROUP BY department;
```

SQL ▾ 1 / 1 1 - 3 of 3

```sql
SELECT department, SUM(sales) AS total_sales
FROM sales_data
GROUP BY department;
```

department	total_sales
HR	2500
IT	4500
Sales	6500

Pandas(Python) Code:

Ass-19.2 > ass-19.2.py > ...

```python
import pandas as pd

# Create sample dataset
data = {
    "department": ["HR", "IT", "HR", "Sales", "IT", "Sales"],
    "sales": [1000, 2000, 1500, 3000, 2500, 3500]
}

df = pd.DataFrame(data)

# Group By + Aggregation
result = df.groupby("department")["sales"].sum().reset_index()

# Display result
print(result)
```

Output:

```
  department  sales
0         HR   2500
1         IT   4500
2      Sales   6500
```

Explanation:

The SQL query groups the data by the department column and calculates the total sales using the SUM() function.

Similarly, in pandas, the groupby() function is used to group the DataFrame by department, and the sum() function computes the total sales for each group. The reset_index() method converts the grouped result into a structured table format.

Both SQL and Pandas follow the same logic:

- Group data based on department
- Apply aggregation (sum of sales)
- Produce identical results

Task-04:

Prompt:

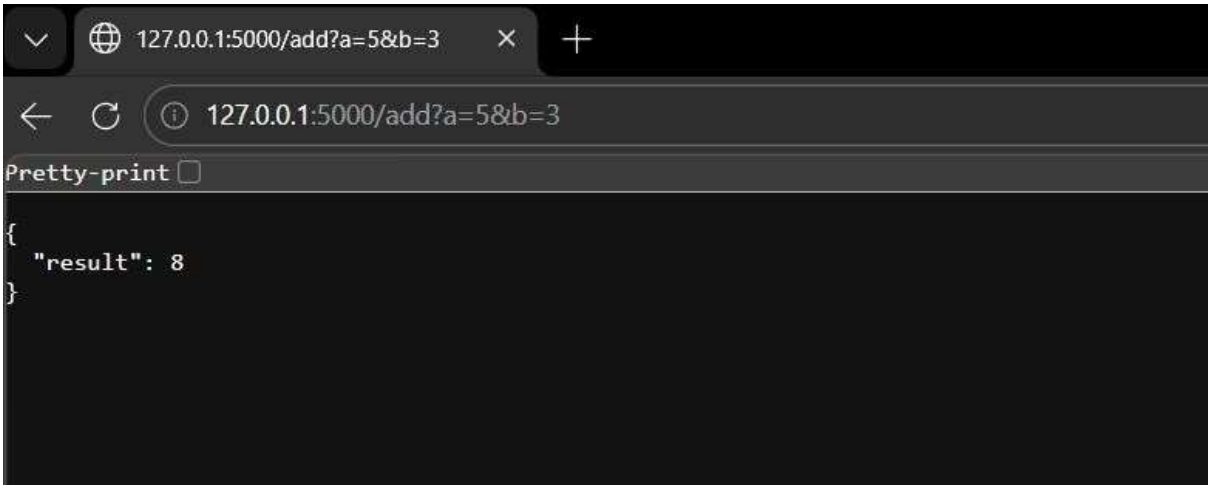
Convert the following Python Flask REST API into an equivalent Java Spring Boot application.

Ensure the same endpoint, functionality, and JSON output behavior.

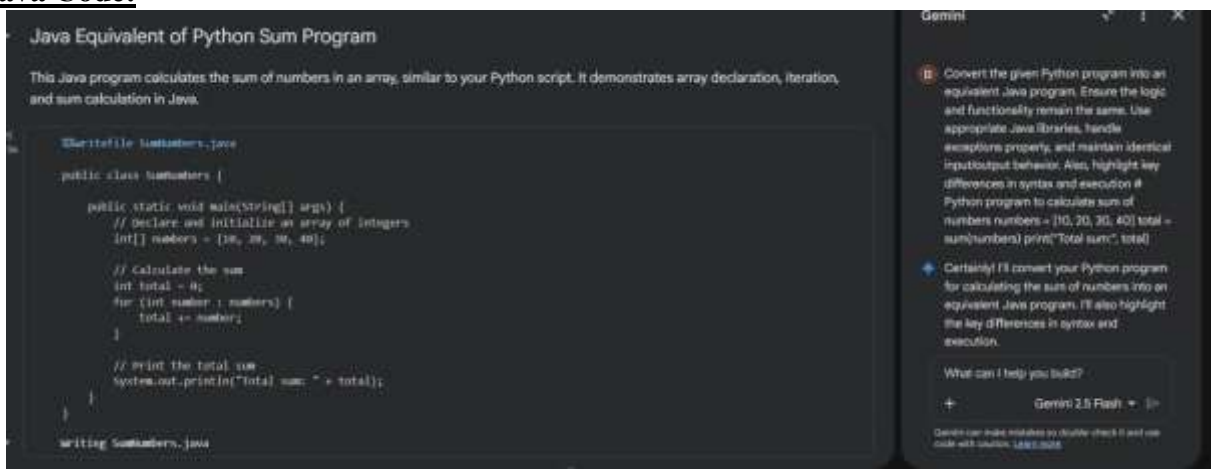
Code:

```python
from flask import Flask, jsonify, request

app = Flask(__name__)

# Home route (to avoid 404 error)
@app.route('/')
def home():
    return "Flask API is running! Use /add?a=5&b=3"

# Add API
@app.route('/add', methods=['GET'])
def add_numbers():
    try:
        a = int(request.args.get('a'))
        b = int(request.args.get('b'))
        return jsonify({"result": a + b})
    except:
        return jsonify({"error": "Invalid input"}), 400

# Run app
if __name__ == '__main__':
    app.run(debug=True)
```

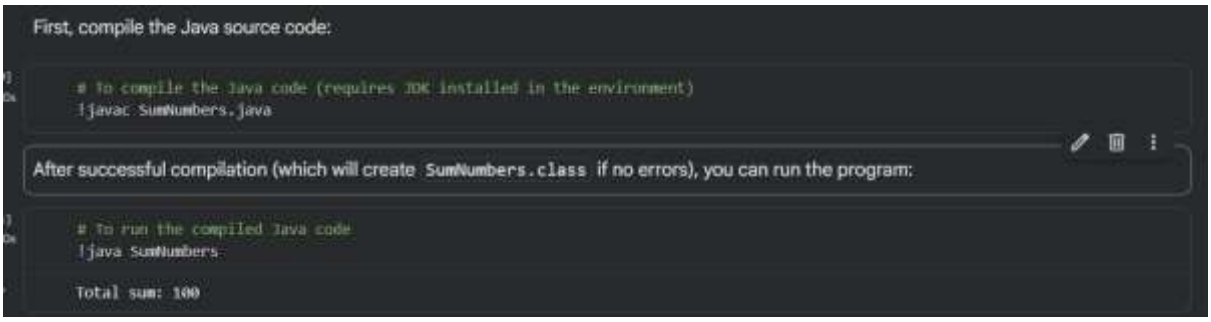
Output:



Java Code:



Output:



Explanation:

Both Python and Java programs calculate the sum of numbers in a list/array using a loop. Each element is added to a total variable, and the final sum is displayed. Python uses a list and simple syntax, while Java uses an array and requires explicit data types. Despite syntax differences, both programs produce the same output.

TASK – 05

Prompt :

Convert the given Python function that checks whether a number is even or odd into an equivalent C++ program. Ensure proper syntax, input handling, and output formatting, and verify that both programs produce the same results.

Function to check even or odd

def check_even_odd(num):

if num % 2 == 0: return "Even"

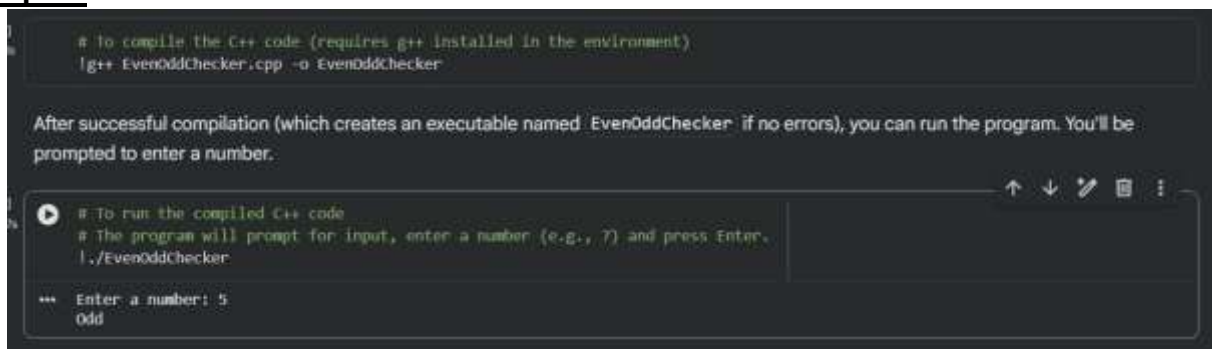
else:

return "Odd"

Input n = int(input("Enter a number: ")) print(check_even_odd(n))

Code :

Output :



Explanation :

The task converts a Python function into an equivalent C++ program.Both programs use the same logic (num % 2) to check even or odd. Proper syntax and input/output handling are ensured in C++. The programs are tested with different inputs to verify correctness. Both versions produce the same output, confirming successful translation.